

to the temperature T , to the stored function value associated with every vertex of the simplex, and we *subtract* a similar random variable from the function value of every new point that is tried as a replacement point. Like the ordinary Metropolis procedure, this method always accepts a true downhill step, but sometimes accepts an uphill one. In the limit $T \rightarrow 0$, this algorithm reduces exactly to the downhill simplex method and converges to a local minimum.

At a finite value of T , the simplex expands to a scale that approximates the size of the region that can be reached at this temperature, and then executes a stochastic, tumbling Brownian motion within that region, sampling new, approximately random, points as it does so. The efficiency with which a region is explored is independent of its narrowness (for an ellipsoidal valley, the ratio of its principal axes) and orientation. If the temperature is reduced sufficiently slowly, it becomes highly likely that the simplex will shrink into that region containing the lowest relative minimum encountered.

As in all applications of simulated annealing, there can be quite a lot of problem-dependent subtlety in the phrase “sufficiently slowly”; success or failure is quite often determined by the choice of annealing schedule. Here are some possibilities worth trying:

- Reduce T to $(1 - \epsilon)T$ after every m moves, where ϵ/m is determined by experiment.
- Budget a total of K moves, and reduce T after every m moves to a value $T = T_0(1 - k/K)^\alpha$, where k is the cumulative number of moves thus far, and α is a constant, say 1, 2, or 4. The optimal value for α depends on the statistical distribution of relative minima of various depths. Larger values of α spend more iterations at lower temperature.
- After every m moves, set T to β times $f_1 - f_b$, where β is an experimentally determined constant of order 1, f_1 is the smallest function value currently represented in the simplex, and f_b is the best function ever encountered. However, never reduce T by more than some fraction γ at a time.

Another strategic question is whether to do an occasional *restart*, where a vertex of the simplex is discarded in favor of the “best-ever” point. (You must be sure that the best-ever point is not one of the other vertices of the simplex when you do this!) We have found problems for which restarts — every time the temperature has decreased by a factor of 3, say — are highly beneficial; we have found other problems for which restarts have no positive, or a somewhat negative, effect.

There is not yet enough practical experience with the method of simulated annealing to say definitively what its place among optimization methods is. The method has several extremely attractive features that are rather unique when compared with other optimization techniques.

First, it is not “greedy,” in the sense that it is not easily fooled by the quick payoff achieved by falling into unfavorable local minima. Provided that sufficiently general reconfigurations are given, it wanders freely among local minima of depth less than about T . As T is lowered, the number of such minima qualifying for frequent visits is gradually reduced.

Second, configuration decisions tend to proceed in a logical order. Changes that cause the greatest energy differences are sifted over when the control parameter T is large. These decisions become more permanent as T is lowered, and attention

then shifts more to smaller refinements in the solution. For example, in the traveling salesman problem with the Mississippi River twist, if λ is large, a decision to cross the Mississippi only twice is made at high T , while the specific routes on each side of the river are determined only at later stages.

The analogies to thermodynamics may be pursued to a greater extent than we have done here. Quantities analogous to specific heat and entropy may be defined, and these can be useful in monitoring the progress of the algorithm toward an acceptable solution [1].

CITED REFERENCES AND FURTHER READING:

- Salamon, P., Sibani, P., and Frost, R. 2002, *Facts, Conjectures, and Improvements for Simulated Annealing* (New York: SIAM Press).
- van Laarhoven, P.J.M., and Aarts, E.H.L. 1987, *Simulated Annealing: Theory and Applications* (Berlin: Springer).
- Kirkpatrick, S., Gelatt, C.D., and Vecchi, M.P. 1983, "Optimization by Simulated Annealing," *Science*, vol. 220, pp. 671–680.[1]
- Kirkpatrick, S. 1984, "Optimization by Simulated Annealing: Quantitative Studies," *Journal of Statistical Physics*, vol. 34, pp. 975–986.[2]
- Vecchi, M.P. and Kirkpatrick, S. 1983, "Global Wiring by Simulated Annealing," *IEEE Transactions on Computer Aided Design*, vol. CAD-2, pp. 215–222.[3]
- Otten, R.H.J.M., and van Ginneken, L.P.P.P. 1989, *The Annealing Algorithm* (Boston: Kluwer) [contains many references to the literature].[4]
- Metropolis, N., Rosenbluth, A., Rosenbluth, M., Teller A., and Teller, E. 1953, "Equations of State Calculations by Fast Computing Machines," *Journal of Chemical Physics*, vol. 21, pp. 1087–1092.[5]
- Lin, S. 1965, "Computer Solutions of the Traveling Salesman Problem," *Bell System Technical Journal*, vol. 44, pp. 2245–2269.[6]
- Numerical Recipes Software 2007, "Code Implementation for the Traveling Salesman Problem," *Numerical Recipes Webnote No. 14*, at <http://www.nr.com/webnotes?14> [7]
- Vanderbilt, D., and Louie, S.G. 1984, "A Monte Carlo Simulated Annealing Approach to Optimization over Continuous Variables," *Journal of Computational Physics*, vol. 56, pp. 259–271.[8]
- Numerical Recipes Software 2007, "Code for Minimization with Simulated Annealing," *Numerical Recipes Webnote No. 15*, at <http://www.nr.com/webnotes?15> [9]
- Bohachevsky, I.O., Johnson, M.E., and Stein, M.L. 1986, "Generalized Simulated Annealing for Function Optimization," *Technometrics*, vol. 28, pp. 209–217.[10]
- Corana, A., Marchesi, M., Martini, C., and Ridella, S. 1987, "Minimizing Multimodal Functions of Continuous Variables with the Simulated Annealing Algorithm," *ACM Transactions on Mathematical Software*, vol. 13, pp. 262–280.[11]
- Bélisle, C.J.P., Romeijn, H.E., and Smith, R.L. 1990, "Hide and Seek: A Simulated Annealing Algorithm for Global Optimization," Technical Report 90–25, Department of Industrial and Operations Engineering, University of Michigan.[12]
- Christofides, N., Mingozi, A., Toth, P., and Sandi, C. (eds.) 1979, *Combinatorial Optimization* (London and New York: Wiley-Interscience) [not simulated annealing, but other topics and algorithms].

10.13 Dynamic Programming

Dynamic programming, or *DP*, is an optimization technique that applies when a known sequence of choices, each with a cost or benefit, is to be made and one wants

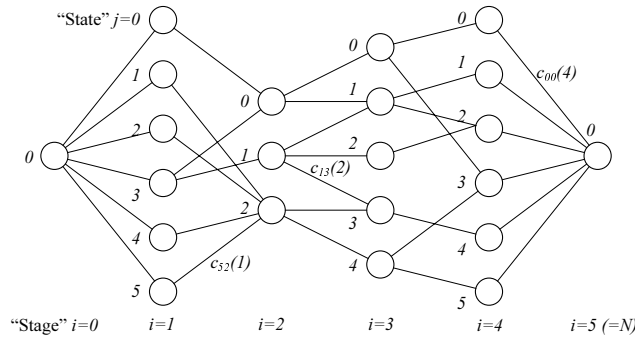


Figure 10.13.1. Canonical dynamical programming problem. It is desired to find the lowest cost path from a starting state to an ending state through $N - 1$ intermediate stages. Each stage is characterized by a set of *states* (not necessarily the same at each stage). An allowed edge between state j in stage i and state k in stage $i + 1$ has a cost denoted $c_{jk}(i)$ (not all labeled in the figure).

to minimize the total cost, or maximize the total benefit, after the sequence has been traversed. More specifically, a problem that is amenable to dynamic programming can be broken up into an ordered series of discrete *stages*, and within each stage a set of discrete *states*. These stages and states form a directed graph (see Figure 10.13.1) that we want to traverse from a given starting state ($i = 0$) to a given ending state ($i = N$). Allowed decisions that take one from state j in stage i to state k in stage $i + 1$ are edges in the graph. Their cost is denoted $c_{jk}(i)$. Without any loss of generality, one can connect all the states in stage i to all the states in stage $i + 1$, but with $c_{jk}(i) = \infty$ for forbidden paths.

Computer science is rich in graph-theoretic problems and algorithms, but only a few of these are within the scope of this book. Dynamic programming is one of these because its basic idea is very simple and its applications are very broad. It is important that you be able to recognize a problem amenable to DP when you see one. In particular, we will use several of the concepts in this section later, in §16.2, when we discuss the estimation of states from probabilistic data, including probabilistic decoding algorithms.

The key idea of dynamic programming is called the Bellman, Dijkstra, or Viterbi algorithm, depending on the field of training of the caller. As shown in Figure 10.13.2, the idea is that one can do a single sweep of a stage-ordered graph from left to right, labeling each vertex by the single number that is the cost of the *best* way of having reached it. (Henceforth we'll take the canonical DP problem as a cost-minimization problem; if your problem is instead a benefit-maximization problem, just use the negative of your benefits as costs.)

When the end state is reached, the global minimum cost of getting to it becomes known. Now, in a single backward pass, we can read off exactly what set of decisions led to this global minimum, by reconstructing which predecessor state was the one actually in the chain that led to the best result. Arriving back at the starting state, our solution is complete.

The art of DP involves, in many cases, the clever organization of the problem to minimize the number of states at each stage, so as to avoid the “curse of dimensionality” (a phrase first used by Bellman in exactly this context). Sometimes the order of the stages is not chronological at all, but merely reflects the decomposition of a

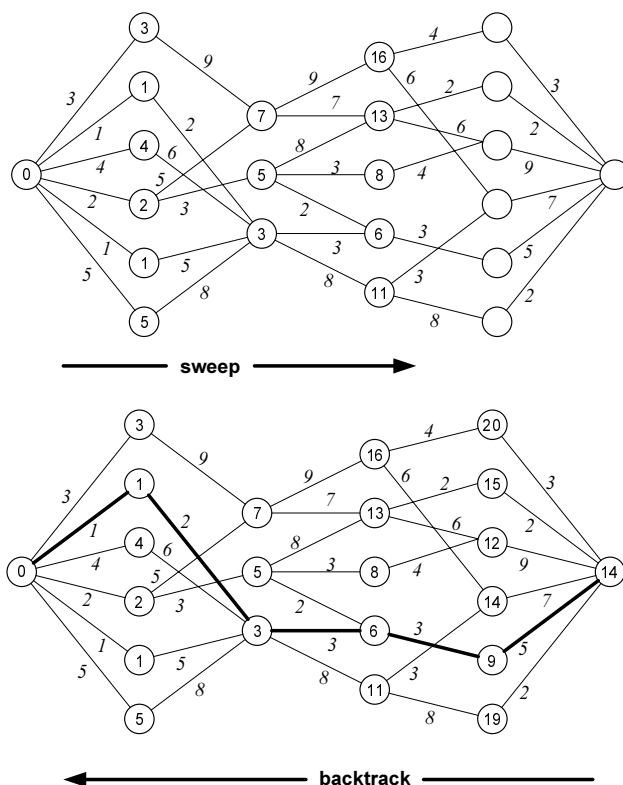


Figure 10.13.2. Two snapshots during the solution of a DP problem by the Bellman-Dijkstra-Viterbi algorithm. Edge costs are given as shown. Top: During the rightward sweep (here not yet complete) each state is labeled by the minimum cost to reach it, as determined solely by the labels of the preceding stage and the connecting edge costs. Bottom: After the rightward sweep is complete, the unique set of edges that produce the global minimum is found by one pass of backtracking.

problem into a convenient form for DP.

Here is a function embodying the Bellman-Dijkstra-Viterbi algorithm. You should consider this function more a precise statement of the algorithm than a production DP code. For example, it simply pushes off to the user function `cost()` the important issue of how to retrieve efficiently the (usually) sparse set of allowed edges that have finite costs. (You might want to consider a hash memory. See §7.6.) Also, this routine loops explicitly over all combinations of states j and k , the origin and destination states in going from stage i to $i + 1$. If you have a problem big enough to need some kind of sparse lookup, then you'll want to change these explicit loops accordingly.

```
VecInt dynpro(const VecInt &nstate,
Doub cost(Int jj, Int kk, Int ii)) {
    Given the vector nstate whose integer values are the number of states in each stage (1
    for the first and last stages), and given a function cost(j,k,i) that returns the cost of
    moving between state j of stage i and state k of stage i+1, this routine returns a vector
    of the same length as nstate containing the state numbers of the lowest cost path. States
    number from 0, and the first and last components of the returned vector will thus always
    be 0.
    const Doub BIG = 1.e99;
```

dynpro.h

```

static const Doub EPS=numeric_limits<Doub>::epsilon();
Int i, j ,k, nstage = nstate.size() - 1;
Doub a,b;
VecInt answer(nstage+1);
if (nstate[0] != 1 || nstate[nstage] != 1)
    throw("One state allowed in first and last stages.");
Doub **best = new Doub*[nstage+1];           Allocate array-of-arrays for storing scores.
best[0] = new Doub[nstate[0]];
best[0][0] = 0.;
for (i=1; i<=nstage; i++) {                  Forward sweep through stages.
    best[i] = new Doub[nstate[i]];
    for (k=0; k<nstate[i]; k++) {
        b = BIG;
        for (j=0; j<nstate[i-1]; j++) {      Find predecessor giving min cost.
            if ((a = best[i-1][j] + cost(j,k,i-1)) < b) b = a;
        }
        best[i][k] = b;
    }
}
answer[nstage] = answer[0] = 0;
for (i=nstage-1; i>0; i--) {                  Backtracking pass.
    k = answer[i+1];
    b = best[i+1][k];
    for (j=0; j<nstate[i]; j++) {             Find a predecessor that gave min.
        Doub temp = best[i][j] + cost(j,k,i);
        if (fabs(b - temp) <= EPS*fabs(temp)) break;
    }
    answer[i] = j;
}
for (i=nstage; i>=0; i--) delete [] best[i];  Cleanup storage.
delete [] best;
return answer;
}

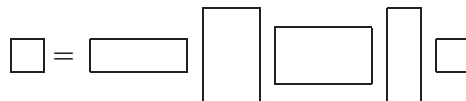
```

10.13.1 Example: Order of Matrix Multiplication

Suppose we have five matrices to multiply, so as to get a result T ,

$$T = ABCDE \quad (10.13.1)$$

The matrices may all have different shapes, as long as the number of columns of a matrix is the same as the number of rows of the matrix immediately to its right. Matrix multiplication is associative, and we can do the multiplies in any order we want; but the total number of scalar multiplications can be quite different, depending on which order is chosen. You should be able to see this in the following figure:



What we want to minimize is the total number of scalar multiplications. In this example, a good choice of “stage” is just how many matrix multiplications have been performed. So the stages and states might look something like this: